Strategies

What a qp strategy is, every part of its grammar, how it is turned into trades, and how it is built and checked in the page. Written from `quant-platform/chart/strategy.py`, `exit_protocol.py`, `order_plan.py`, `store.py`, `parity.py` and the strategy drawer of `chart/static/index.html`.

For	Read
the indicators a strategy can use	LIBRARY.md, CATALOG.md
running a strategy over many days	BACKTEST.md
the page and its strategy drawer	PAGE.md
the five SMB scalps built in this grammar	SCALPS.md, SCALPS_SPEC.md

---

The one rule

A strategy is evaluated over **the same verified primitive values the chart draws** (`compare_server.overlay_arrays`). The chart preview, the backtest and the alerts all call the same `strategy.evaluate()`. There is no second evaluator, so a signal on the chart is the signal the backtest trades.

A strategy document

```
{
  "name": "OR + VWAP 09:35 (Long)",
  "description": "…",
  "side": "long",
  "stage": "ready",
  "tools": ["T2"],
  "entry": { "logic": "AND", "rules": [ … ] },
  "exit":  { "logic": "OR",  "rules": [ … ] },
  "risk":  { "sl": {…}, "tp": {…}, "targets": [ … ], "window_start": 935, … },
  "exit_plan": { … }
}
```

Field	Meaning
`side`	`long` or `short`
`stage`	`development` (new strategies start here) or `ready` (finished). Set by hand; never earned automatically
`tools`	the screener tools whose registers the strategy is meant for; refused on save if a tool id does not exist. Used as the backtest’s default universe
`entry`, `exit`	rule groups (below)
`risk`	stops, targets, discipline and size gates (below)
`exit_plan`	optional: the exit declared leg by leg (below)

Operands: the values a rule compares

Kind	Example	Gives
`primitive`	`{"kind":"primitive","key":"ma.ema","source":"close","params":{"length":9}}`	a verified indicator; `sub` picks one line of a multi-line primitive (floor pivot `R1`, Bollinger `upper`, dynamic S/R `sr3`)
`price`	`{"kind":"price","field":"close"}`	`close`, `open`, `high`, `low`, `hl2`, `hlc3`, `ohlc4`, `volume`
`time`	`{"kind":"time","field":"minutes"}`	the bar's New York time: `minutes` since midnight, `hhmm`, `hour`, `minute`
`const`	`{"kind":"const","value":1.5}`	a number
`expr`	`{"kind":"expr","op":"div","a":…,"b":…}`	arithmetic on two operands, nested freely: `add`, `sub`, `mul`, `div` (÷0 gives no value). E.g. body % = body ÷ range × 100; move in ATR = (close − open) ÷ ATR
`trade`	`{"kind":"trade","field":"pnl_pct"}`	**exit rules only**: the open

Kind	Example	Gives
		position's `entry` price, `bars` held, unrealised `pnl_pct`; each trade sees its own

Any operand may also carry:

Modifier	Effect
`offset: n`	the value `n` bars ago (`close > high[1]`)
`hold: true`	carry the last real value forward until it changes (Pine's `fixnan`), applied **before** the offset. Turns a sparse primitive such as a pivot high into a fixed level that a later bar can retest
`symbol`	read the operand from another symbol (e.g. SPY), aligned bar by bar without look-ahead

A primitive on a coarser timeframe (daily ATR, average volume) only shows the last **completed** higher-timeframe bar: no same-day look-ahead.

Rules

```
{ "left": <operand>, "op": "gt", "right": <operand>,
  "op_params": {…}, "for_bars": 3, "within_bars": 5, "offset": 1 }
```

Operators

op	True when
`gt` `lt` `ge` `le`	left is above / below / at or above / at or below right
`eq` `neq`	equal / not equal (floating-point tolerant)
`gt_pct` `lt_pct`	above / below by **at least** `pct` % of the right side's size: `L ≥ R +
`cross_above` `cross_below`	the previous bar was at or below (above) and this bar is above (below)
`rising` `falling`	over `lookback` bars (default **12**): the line ends above (below) where it started; optionally by at least `min_pct` %; and at least `consistency` (default **75 %**) of the window's non-zero bar-to-bar changes point that way. Flat bars do not count against it, so a step line that moves rarely but always one way qualifies; a flat or choppy line is neither. `right` is ignored
`bounce_up` `bounce_down`	the price bar bounces off the right-hand level: its wick comes within `tol_pct` of it (default **0.15 %**) and the bar closes back on its side, in the top 60 % of its own range for a bounce up (`close_pos`)

Any missing value (warm-up, outside a session) makes the rule false.

Timing on a rule

Field	True when
`for_bars: N`	it held on **all** of the last N bars (on a cross: the cross happened N−1 bars ago and the state survived since)
`within_bars: N`	it held on **any** of the last N bars
`offset: N`	the rule's result is shifted: it was true N bars **ago** (shifts the result; an operand offset shifts a value)

Groups

```
{ "logic": "AND", "rules": [ rule, rule, { "logic": "OR", "rules": [ … ] } ] }
```

`logic`	**True when**
`AND`	all its rules hold
`OR`	any holds
`ATLEAST` (`k`)	at least `k` of them hold (scoring, “min core / min extra”); `k` below 1 counts as 1
`THEN` (`window`)	its steps happen **in order**, each within `window` bars (default **10**) of the previous: e.g. *above the 13-EMA* THEN *bounces up off the 13-EMA*

Groups nest to any depth: `A and B and (C or D) and (E or F)`.

From rules to trades

Entry

- `entry_mode: edge` (default): one entry per stretch of bars on which the entry group is true; a THEN sequence completing again is a new stretch.
- `entry_mode: status`: any bar on which it is true while flat (so it can re-enter after a stop while the setup still holds).

Exit, in a fixed priority

While in a position, on each bar:

1. **Stop** first. With a single target, if the stop and the target fall in the same bar, the stop wins (conservative). With scale-out legs, a leg target the bar traded through is banked first and the stop applies to what is left, as a real bracket does; `stop_first: true` makes the stop win instead.
2. **Target** (or the next scale-out leg).
3. **The exit rule**, as a status: it exits on any bar where the exit group is true, even if it was already true before the entry. An exit rule that uses a `trade` operand is evaluated per trade.
4. **End of day**, when the session rules force it: at that bar’s close, reason `eod`.

A stop hit while the trade is in profit (an anchored or trailing stop that has moved) is recorded as `trail`.

The risk block

Field	Meaning
`sl`, `tp`	`{type, value}` with `type` = `pct` (% of entry), `atr` (× ATR 14 at entry), `points` (dollars)
	or `{type: "prim", anchor: <operand>, value: <pct beyond>}`: a level that **trails** an indicator bar by bar, shifted to the protective side (0.2 % under the 9-EMA for a long)
	`freeze: true` on a `prim` stop or target: read the level once at the signal bar and hold it for the trade (“2 cents under the low of the turn”)
`targets`	scale-out legs: each `{fraction, r_multiple}` or `{fraction, tp: {…}}`; what the legs do not take stays until the stop, the exit rule or the close
`window_start`, `window_end`	`hhmm`: entries only inside this window
`max_entries_per_day`	per symbol
`cooldown_bars`	after an exit, no new entry for this many bars (the clock resets each day)
`min_hold_bars`	the exit rule is ignored for this many bars after entry
`max_stop_pct`	refuse an entry whose stop is further than N %
`min_stop_pct`	refuse an entry whose stop is closer than N % (it cannot pay for its own costs)
`min_target_usd`	refuse an entry whose nearest target is less than $X from the fill
`stop_first`	when one bar touches both the stop and a leg target, assume the stop filled (default off)
`entry_mode`	`edge` or `status` (above)
`breakeven_after_target`	**reported on the order plan, not simulated** (Q-N7)

Session rules (prop-firm style)

Passed with a run (`rules`): `rth_entries` with `entry_start` (default 09:30) and `entry_cutoff` (default 15:50) to allow opening only in regular hours; `eod_close` to flatten every position at the day’s last bar; and `max_entries_per_day`. With a next-bar fill, a signal whose fill would land outside the window is dropped.

Fill models: the price a trade gets

Explicit on every run; **an unknown model is an error, never a silent fallback**.

`fill`	**Entry fills at**	**Stop and targets measured from**	**Use**
`close`	the signal bar's close	the same close	chart preview; optimistic by about a spread
`next_open`	the next bar's open	**the fill** (re-prices R from the fill, which a bracket order cannot do)	comparison
`desk` (backtest default)	the next bar's open	**the decision bar's close**, as a bracket order really is priced	the honest backtest
`live`	(reported as the decision price)	the decision bar's close	`desk` with the future removed, so a live pick and a backtest trade come from the same bar. Built for the live decision of the removed trading module; **nothing uses it now**: the alerts evaluate with `close` (Q-N9)

Why `desk` exists, from the code's own record: on 2026-08-19 a WULF short was decided at 15.37, filled at 15.24, stop at 15.74. The plan said 2.00 R; priced from the fill, the same trade was 1.31 R. `desk` books the fill at the open and keeps every level where the decision put it.

With a next-bar fill: a signal on the last bar makes no trade; a stop fills at the level on a touch, but at the **open** when the bar gaps through it (the gap is charged); a target fills at its level.

The exit plan and the order

The exit is declared, never inferred (`exit_protocol.py`). Every strategy reports the same structure, written or derived from `risk`:

```
exit_plan = {
  shape: "2 SL / 2 TP + runner",
  legs: [ {fraction 0.5, sl, tp}, {fraction 0.25, sl, tp} ],
  runner: {fraction 0.25, manage: "eod" | "manual"},
  ok, errors, warnings, derived
}
```

Two rules make the quantity correct: **every leg has its own stop** (a shared stop is filled in from `risk.sl`, explicitly), and **the fractions sum to exactly 1**. A plan that sums to 0.9 means a tenth of every position is never ordered.

The order (`order_plan.py`) is one pure function: a trade and an account in, the orders out, or the reason there are none.

1. Risk budget: `risk_usd`, or `risk_pct` of the account size.
2. Shares = budget ÷ risk per share.
3. Caps, in order: the position cap (% of the account), the account ratio, the per-order value cap, the money left.
4. Below `min_shares`: nothing is sent.
5. One bracket per leg in whole shares; the odd shares go to the runner (or the last leg).
6. Prices in whole cents, rounded towards the entry.

The backtest books exactly these orders.

Building and checking one

In the page's ⚡ **Strategy** drawer (PAGE.md):

Tool	Does
▶ **Evaluate**	runs the strategy over the chart's window: entry ▲ and exit ▼ markers, the stop and target drawn as the simulation used them (a fixed stop flat, an anchored one trailing its line), the indicators the rules read, whether entry and exit are true **now**, and a quick preview (win rate, return, exits by reason). Can replay a backtest's exact fill and session settings
🔍 **test** on any row	marks **every** bar where that one rule or group holds, draws the lines it reads, gives the left side's current value and how often it is true ("rising on 22 % of bars, TRUE now")
`GET /api/strategy/explain`	the **signal funnel**: on one symbol and day, every entry step's true-bar count; the step at zero is the blocker
`GET /api/strategy/explain_scan`	the funnel over a whole register day, ranking who came closest to firing
`parity.py`	is the strategy as saved today the one a backtest ran? (every backtest stores a frozen copy of the strategies it ran)

The working rule, from `chart/README.md`: **never trust a piece you have not seen**. Plot the number, test the condition, then compose.

Where strategies live

Copy	Where	Role
the database	`~/.qp/platform.db`, table `strategies`	the working copy
JSON copies	`~/.qp/strategies/*.json`, one per strategy, written on every save	read back only when the database has **no** strategies (wiped or new box); never merged into one that has rows
seeds	`chart/seeds/*.json`, in git	synced into the database at start. Canonical seeds track the bundle; a seed marked `_keep_user_edits`, or one edited in the browser, keeps the stored copy. One bad seed is skipped and named, never allowed to stop the rest
backtest snapshots	every run stores the strategies it ran	`tools/recover_strategies.py` can read a lost strategy back from them, or from SQLite's free pages

The bundled seeds:

Seed	Side	Stage	Window
Test	long	ready	09:30–11:30
OR + VWAP 09:35 (Long / Short)	both	ready	09:35
T2 10:00 VWAP Extension (Long / Short)	both	ready	10:00
RubberBand Scalp (Long / Short), Second Chance, Back$ide, HitchHiker, Fashionably Late (SCALPS.md)	long (one short)	development	per scalp
PM Breakout (2m)	long	development	09:30–10:00
PML breakout (the user's own, restore-only)	short	development	09:40–10:10

The three ready strategies also exist as Pine scripts for TradingView's Strategy Tester (PINE.md).